

vLLM专题（九）- 量化

 大模型专题系列 专栏收录该内容

¥49.90
¥99.00

111 篇文章

已订阅

8折续

 您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益 >

1.支持的硬件

下表展示了 vLLM 中各种量化实现与不同硬件平台的兼容性：

Implementation	Volta	Turing	Ampere	Ada	Hopper	AMD GPU	Intel GPU	x86 CPU	AWS Inferentia	Google TPU
AWQ	✗	✓	✓	✓	✓	✗	✓	✓	✗	✗
GPTQ	✓	✓	✓	✓	✓	✗	✓	✓	✗	✗
Marlin (GPTQ/AWQ/FP8)	✗	✗	✓	✓	✓	✗	✗	✗	✗	✗
INT8 (W8A8)	✗	✓	✓	✓	✓	✗	✗	✓	✗	✗
FP8 (W8A8)	✗	✗	✗	✓	✓	✓	✗	✗	✗	✗
AQLM	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗
bitsandbytes	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗
DeepSpeedFP	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗
GGUF	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗

Volta 指 SM 7.0，Turing 指 SM 7.5，Ampere 指 SM 8.0/8.6，Ada 指 SM 8.9，Hopper 指 SM 9.0。

☑ 表示量化方法在指定硬件上受支持。

☐ 表示量化方法在指定硬件上不受支持。

注意

此兼容性图表可能会随着 vLLM 的不断发展和对不同硬件平台及量化方法的支持扩展而发生变化。

有关硬件支持和量化方法的最新信息，请参考 [vllm/model_executor/layers/quantization](#) 或咨询 vLLM 开发团队。

2. 自动 AWQ (AutoAWQ)

要创建一个新的 4 位量化模型，您可以利用 AutoAWQ。量化将模型的精度从 FP16 降低到 INT4，从而有效减少约 70% 的文件大小。主要优势是降低延迟和内存使用。

您可以通过安装 AutoAWQ 或选择 Huggingface 上的 400 多个模型之一来量化自己的模型。

```
pip install autoawq
```

安装 AutoAWQ 后，您就可以开始量化模型了。以下是如何量化 `mistralai/Mistral-7B-Instruct-v0.2` 的示例：

```
from awq import AutoAWQForCausalLM
from transformers import AutoTokenizer

model_path = 'mistralai/Mistral-7B-Instruct-v0.2'
quant_path = 'mistral-instruct-v0.2-awq'
quant_config = {
    "zero_point": True, "q_group_size": 128, "w_bit": 4, "version": "GEMM" }

# Load model
model = AutoAWQForCausalLM.from_pretrained(
    model_path, **{
        "low_cpu_mem_usage": True, "use_cache": False}
)
tokenizer = AutoTokenizer.from_pretrained(model_path, trust_remote_code=True)

# Quantize
model.quantize(tokenizer, quant_config=quant_config)

# Save quantized model
model.save_quantized(quant_path)
tokenizer.save_pretrained(quant_path)

print(f'Model is quantized and saved at "{
    quant_path}")')
```

要使用 vLLM 运行 AWQ 模型，您可以使用 `TheBloke/Llama-2-7b-Chat-AWQ`，并执行以下命令：

```
python examples/offline_inference/llm_engine_example.py --model TheBloke/Llama-2-7b-Chat-AWQ --quantization awq
```

AWQ模型也可以通过LLM入口点直接支持。

```

from vllm import LLM, SamplingParams

# Sample prompts.
prompts = [
    "Hello, my name is",
    "The president of the United States is",
    "The capital of France is",
    "The future of AI is",
]

# Create a sampling params object.
sampling_params = SamplingParams(temperature=0.8, top_p=0.95)

# Create an LLM.
llm = LLM(model="TheBloke/Llama-2-7b-Chat-AWQ", quantization="AWQ")
# Generate texts from the prompts. The output is a list of RequestOutput objects
# that contain the prompt, generated text, and other information.
outputs = llm.generate(prompts, sampling_params)
# Print the outputs.
for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {
        prompt!r}, Generated text: {
        generated_text!r}")

```

3. BitsAndBytes

vLLM现在支持BitsAndBytes，以实现更高效的模型推理。BitsAndBytes通过量化模型来减少内存使用并提高性能，同时不会显著牺牲准确性。与其他量化方法相比，BitsAndBytes无需通过输入数据对量化模型进行校准。

以下是使用vLLM与BitsAndBytes的步骤。

```
pip install bitsandbytes>=0.45.0
```

vLLM读取模型的配置文件，支持实时量化和预量化检查点。

您可以在 <https://huggingface.co/models?other=bitsandbytes> 上找到bitsandbytes量化的模型。通常，这些仓库会有一个config.json文件，其中包含quantization_config部分。

3.1. 读取量化检查点

```
from vllm import LLM
import torch
# unsloth/tinyllama-bnb-4bit is a pre-quantized checkpoint.
model_id = "unsloth/tinyllama-bnb-4bit"
llm = LLM(model=model_id, dtype=torch.bfloat16, trust_remote_code=True, \
quantization="bitsandbytes", load_format="bitsandbytes")
```

3.2. 实时量化：加载为4位量化

```
from vllm import LLM
import torch
model_id = "huggyllama/llama-7b"
llm = LLM(model=model_id, dtype=torch.bfloat16, trust_remote_code=True, \
quantization="bitsandbytes", load_format="bitsandbytes")
```

3.3. OpenAI兼容服务器

将以下内容附加到您的4位模型参数中：

```
--quantization bitsandbytes --load-format bitsandbytes
```

4. GGUF

警告

请注意，vLLM中的GGUF支持目前仍处于高度实验阶段，且尚未经过优化，可能与其他功能不兼容。目前，您可以使用GGUF来减少内存占用。如果遇到任何问题，请向vLLM团队报告。

警告

目前，vLLM仅支持加载单文件GGUF模型。如果您有多个文件的GGUF模型，您可以使用gguf-split工具将它们合并为单文件模型。

要使用vLLM运行GGUF模型，您可以下载并使用来自TheBloke/TinyLlama-1.1B-Chat-v1.0-GGUF的本地GGUF模型，使用以下命令：

```
wget https://huggingface.co/TheBloke/TinyLlama-1.1B-Chat-v1.0-GGUF/resolve/main/tinyllama-1.1b-chat-v1.0.Q4_K_M.gguf
# We recommend using the tokenizer from base model to avoid long-time and buggy tokenizer conversion.
vllm serve ./tinyllama-1.1b-chat-v1.0.Q4_K_M.gguf --tokenizer TinyLlama/TinyLlama-1.1B-Chat-v1.0
```

您还可以添加 `--tensor-parallel-size 2` 来启用使用2个GPU的张量并行推理：

```
# We recommend using the tokenizer from base model to avoid long-time and buggy tokenizer conversion.  
vllm serve ./tinylama-1.1b-chat-v1.0.Q4_K_M.gguf --tokenizer TinyLlama/TinyLlama-1.1B-Chat-v1.0 --tensor-parallel-size 2
```

警告

我们建议使用基础模型的分词器，而不是GGUF模型。因为从GGUF转换分词器既耗时又不稳定，尤其是一些词汇量较大的模型。

您也可以通过LLM入口点直接使用GGUF模型：

```

from vllm import LLM, SamplingParams

# In this script, we demonstrate how to pass input to the chat method:
conversation = [
    {
        "role": "system",
        "content": "You are a helpful assistant"
    },
    {
        "role": "user",
        "content": "Hello"
    },
    {
        "role": "assistant",
        "content": "Hello! How can I assist you today?"
    },
    {
        "role": "user",
        "content": "Write an essay about the importance of higher education.",
    },
]

# Create a sampling params object.
sampling_params = SamplingParams(temperature=0.8, top_p=0.95)

# Create an LLM.
llm = LLM(model="./tinyllama-1.1b-chat-v1.0.Q4_K_M.gguf",
          tokenizer="TinyLlama/TinyLlama-1.1B-Chat-v1.0")

# Generate texts from the prompts. The output is a list of RequestOutput objects
# that contain the prompt, generated text, and other information.
outputs = llm.chat(conversation, sampling_params)

# Print the outputs.
for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {
        prompt!r}, Generated text: {
        generated_text!r}")

```

5. INT4 W4A16

vLLM支持将权重量化为INT4，以节省内存并加速推理。这种量化方法特别适用于减少模型大小，并在每秒查询量（QPS）较低的工作负载中保持低延迟。

请访问HF集合，查看适用于vLLM的流行LLM的量化INT4检查点，随时使用。

注意

INT4计算仅在计算能力大于8.0的NVIDIA GPU上支持（如Ampere、Ada Lovelace、Hopper、Blackwell）。

5.1. 前提条件

要在vLLM中使用INT4量化，您需要安装llm-compressor库：

```
pip install llmcompressor
```

5.2. 量化过程

量化过程包括四个主要步骤：

1. 加载模型
2. 准备校准数据
3. 应用量化
4. 在vLLM中评估准确性

5.2.1 加载模型

使用标准的transformers AutoModel类加载您的模型和分词器：

```
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL_ID = "meta-llama/Meta-Llama-3-8B-Instruct"
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID, device_map="auto", torch_dtype="auto",
)
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
```

5.2.2 准备校准数据

在将权重量化为INT4时，您需要样本数据来估计权重更新和校准的缩放因子。最好使用与您的部署数据相匹配的校准数据。对于通用指令调优模型，您可以使用像ultrachat这样的数据集：

```

from datasets import load_dataset

NUM_CALIBRATION_SAMPLES = 512
MAX_SEQUENCE_LENGTH = 2048

# Load and preprocess the dataset
ds = load_dataset("HuggingFaceH4/ultrachat_200k", split="train_sft")
ds = ds.shuffle(seed=42).select(range(NUM_CALIBRATION_SAMPLES))

def preprocess(example):
    return {
        "text": tokenizer.apply_chat_template(example["messages"], tokenize=False)}
ds = ds.map(preprocess)

def tokenize(sample):
    return tokenizer(sample["text"], padding=False, max_length=MAX_SEQUENCE_LENGTH, truncation=True, add_special_tokens=False)
ds = ds.map(tokenize, remove_columns=ds.column_names)

```

5.2.3 应用量化

现在，应用量化算法：

```

from llmcompressor.transformers import oneshot
from llmcompressor.modifiers.quantization import GPTQModifier
from llmcompressor.modifiers.smoothquant import SmoothQuantModifier

# Configure the quantization algorithms
recipe = GPTQModifier(targets="Linear", scheme="W4A16", ignore=["lm_head"])

# Apply quantization
oneshot(
    model=model,
    dataset=ds,
    recipe=recipe,
    max_seq_length=MAX_SEQUENCE_LENGTH,
    num_calibration_samples=NUM_CALIBRATION_SAMPLES,
)

# Save the compressed model
SAVE_DIR = MODEL_ID.split("/")[1] + "-W4A16-G128"
model.save_pretrained(SAVE_DIR, save_compressed=True)
tokenizer.save_pretrained(SAVE_DIR)

```

这个过程会创建一个W4A16模型，其中权重量化为4位整数。

5.2.4 评估准确性

量化后，您可以在vLLM中加载并运行模型：

```
from vllm import LLM
model = LLM("./Meta-Llama-3-8B-Instruct-W4A16-G128")
```

为了评估准确性，您可以使用lm_eval：

```
$ lm_eval --model vllm \
  --model_args pretrained="./Meta-Llama-3-8B-Instruct-W4A16-G128",add_bos_token=true \
  --tasks gsm8k \
  --num_fewshot 5 \
  --limit 250 \
  --batch_size 'auto'
```

注意

量化模型对bos令牌的存在可能较为敏感。运行评估时，请确保包含 `add_bos_token=True` 参数。

5.3. 最佳实践

- 从512个样本开始作为校准数据，如果准确性下降，则增加样本量。
- 确保校准数据包含高多样性的样本，以防止过拟合特定的用例。
- 使用2048的序列长度作为起始点。
- 使用模型训练时的聊天模板或指令模板。
- 如果您已对模型进行了微调，可以考虑使用部分训练数据作为校准数据。
- 调整量化算法的关键超参数：
 - `dampening_frac` 设置GPTQ算法的影响程度。较低的值可以提高准确性，但可能导致数值不稳定，从而导致算法失败。
 - `actorder` 设置激活顺序。在压缩层权重时，通道量化的顺序非常重要。将 `actorder="weight"` 设置可以在不增加延迟的情况下提高准确性。

以下是您可以根据自己的用例调整的量化配方示例：

```

from compressed_tensors.quantization import (
    QuantizationArgs,
    QuantizationScheme,
    QuantizationStrategy,
    QuantizationType,
)
recipe = GPTQModifier(
    targets="Linear",
    config_groups={

        "config_group": QuantizationScheme(
            targets=["Linear"],
            weights=QuantizationArgs(
                num_bits=4,
                type=QuantizationType.INT,
                strategy=QuantizationStrategy.GROUP,
                group_size=128,
                symmetric=True,
                dynamic=False,
                actorder="weight",
            ),
        ),
    },
    ignore=["lm_head"],
    update_size=NUM_CALIBRATION_SAMPLES,
    dampening_frac=0.01
)

```

5.4. 故障排除与支持

如果遇到任何问题或有功能请求，请在vllm-project/llm-compressor的GitHub仓库中提交问题。llm-compressor中的完整INT4量化示例可在此处找到。

6. INT8 W8A8

vLLM支持将权重和激活值量化为INT8，以节省内存并加速推理。这种量化方法特别适用于减少模型大小，同时保持良好的性能。

请访问HF集合，查看适用于vLLM的流行LLM的量化INT8检查点。

注意

INT8计算支持在计算能力大于7.5的NVIDIA GPU上（如Turing、Ampere、Ada Lovelace、Hopper、Blackwell）。

6.1. 前提条件

要在vLLM中使用INT8量化，您需要安装llm-compressor库：

```
pip install llmcompressor
```

6.2. 量化过程

量化过程包括四个主要步骤：

1. 加载模型
2. 准备校准数据
3. 应用量化
4. 在vLLM中评估准确性

6.2.1 加载模型

使用标准的transformers AutoModel类加载您的模型和分词器：

```
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL_ID = "meta-llama/Meta-Llama-3-8B-Instruct"
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID, device_map="auto", torch_dtype="auto",
)
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
```

6.2.2 准备校准数据

在将激活值量化为INT8时，您需要样本数据来估计激活缩放因子。最好使用与您的部署数据相匹配的校准数据。对于通用指令调优模型，您可以使用像ultrachat这样的数据集：

```

from datasets import load_dataset

NUM_CALIBRATION_SAMPLES = 512
MAX_SEQUENCE_LENGTH = 2048

# Load and preprocess the dataset
ds = load_dataset("HuggingFaceH4/ultrachat_200k", split="train_sft")
ds = ds.shuffle(seed=42).select(range(NUM_CALIBRATION_SAMPLES))

def preprocess(example):
    return {
        "text": tokenizer.apply_chat_template(example["messages"], tokenize=False)}
ds = ds.map(preprocess)

def tokenize(sample):
    return tokenizer(sample["text"], padding=False, max_length=MAX_SEQUENCE_LENGTH, truncation=True, add_special_tokens=False)
ds = ds.map(tokenize, remove_columns=ds.column_names)

```

6.2.3 应用量化

现在，应用量化算法：

```

from llmcompressor.transformers import oneshot
from llmcompressor.modifiers.quantization import GPTQModifier
from llmcompressor.modifiers.smoothquant import SmoothQuantModifier

# Configure the quantization algorithms
recipe = [
    SmoothQuantModifier(smoothing_strength=0.8),
    GPTQModifier(targets="Linear", scheme="W8A8", ignore=["lm_head"]),
]

# Apply quantization
oneshot(
    model=model,
    dataset=ds,
    recipe=recipe,
    max_seq_length=MAX_SEQUENCE_LENGTH,
    num_calibration_samples=NUM_CALIBRATION_SAMPLES,
)

# Save the compressed model
SAVE_DIR = MODEL_ID.split("/")[1] + "-W8A8-Dynamic-Per-Token"
model.save_pretrained(SAVE_DIR, save_compressed=True)
tokenizer.save_pretrained(SAVE_DIR)

```

这个过程会创建一个W8A8模型，其中权重和激活值量化为8位整数。

6.2.4 评估准确性

量化后，您可以在vLLM中加载并运行模型：

```
from vllm import LLM
model = LLM("./Meta-Llama-3-8B-Instruct-W8A8-Dynamic-Per-Token")
```

为了评估准确性，您可以使用lm_eval：

```
$ lm_eval --model vllm \
  --model_args pretrained="./Meta-Llama-3-8B-Instruct-W8A8-Dynamic-Per-Token",add_bos_token=True \
  --tasks gsm8k \
  --num_fewshot 5 \
  --limit 250 \
  --batch_size 'auto'
```

注意

量化模型对bos token的存在可能较为敏感。运行评估时，请确保包含 `add_bos_token=True` 参数。

6.3. 最佳实践

- 从512个样本开始作为校准数据（如果准确性下降，则增加样本量）
- 使用2048的序列长度作为起始点
- 使用模型训练时的聊天模板或指令模板
- 如果您已对模型进行了微调，可以考虑使用部分训练数据作为校准数据

6.4. 故障排除与支持

如果遇到任何问题或有功能请求，请在vllm-project/llm-compressor的GitHub仓库中提交问题。

7. FP8 W8A8

vLLM支持使用硬件加速在GPU上进行FP8（8位浮点点数）权重和激活值量化，支持的GPU包括Nvidia H100和AMD MI300x。目前，只有Hopper和Ada Lovelace GPU官方支持W8A8。Ampere GPU支持W8A16（仅权重FP8），使用Marlin内核。使用FP8量化模型可以将模型内存需求减少2倍，并将吞吐量提高最多1.6倍，同时对准确性的影响最小。

请访问HF集合，查看适用于vLLM的流行LLM的量化FP8检查点。

FP8类型通常在硬件中支持，具有两种不同的表示方式，每种表示方式在不同场景中有不同的用途：

- E4M3：由1个位的符号位、4个位的指数位和3个位的尾数位组成。它可以存储的值范围为 ± 448 和nan。
- E5M2：由1个位的符号位、5个位的指数位和2个位的尾数位组成。它可以存储的值范围为 ± 57344 ， $\pm$ -无穷大和nan。增加动态范围的权衡是存储值的精度较低。

注意

FP8计算支持在计算能力大于8.9的NVIDIA GPU上（Ada Lovelace、Hopper）。FP8模型将在计算能力大于8.0的GPU（Ampere）上运行，作为仅权重的W8A16，利用FP8 Marlin。

7.1. 快速启动在线动态量化

使用vLLM，您可以将原始精度的BF16/FP16模型动态量化为FP8，而无需任何校准数据。您可以通过在命令行中指定 `--quantization="fp8"` 或在LLM构造函数中设置 `quantization="fp8"` 来启用该功能。

在此模式下，所有Linear模块（除了最终的lm_head）将其权重量化为FP8_E4M3精度，并使用每个张量的缩放因子。在每次前向传递过程中，激活值的最小值和最大值会被计算出来，以提高高精度的动态每张量缩放因子。因此，在此模式下，延迟改进是有限的。

```
from vllm import LLM
model = LLM("facebook/opt-125m", quantization="fp8")
# INFO 06-10 17:55:42 model_runner.py:157] Loading model weights took 0.1550 GB
result = model.generate("Hello, my name is")
```

警告

目前，我们在量化为8位之前，以原始精度加载模型，因此您需要足够的内存来加载整个模型。

7.2. 安装

要使用vLLM生成高性能的FP8量化模型，您需要安装llm-compressor库：

```
pip install llmcompressor
```

7.3. 量化过程

量化过程包括三个主要步骤：

1. 加载模型
2. 应用量化
3. 在vLLM中评估准确性

7.3.1 加载模型

使用标准的transformers AutoModel类加载您的模型和分词器：

```
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL_ID = "meta-llama/Meta-Llama-3-8B-Instruct"
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID, device_map="auto", torch_dtype="auto",
)
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
```

7.3.2 应用量化

对于FP8量化，我们可以通过简单的RTN量化恢复准确性。我们建议针对所有Linear层使用FP8_DYNAMIC方案，该方案使用：

- 对权重进行静态的、按通道量化
- 对激活值进行动态的、按令牌量化

由于简单的RTN不需要数据来进行权重量化，而激活值是动态量化的，因此我们在此量化流程中不需要任何校准数据。

```
from llmcompressor.transformers import oneshot
from llmcompressor.modifiers.quantization import QuantizationModifier

# Configure the simple PTQ quantization
recipe = QuantizationModifier(
    targets="Linear", scheme="FP8_DYNAMIC", ignore=["lm_head"])

# Apply the quantization algorithm.
oneshot(model=model, recipe=recipe)

# Save the model.
SAVE_DIR = MODEL_ID.split("/")[1] + "-FP8-Dynamic"
model.save_pretrained(SAVE_DIR)
tokenizer.save_pretrained(SAVE_DIR)
```

7.3.3 评估准确性

安装vllm和lm-evaluation-harness：

```
pip install vllm lm-eval==0.4.4
```

在vLLM中加载并运行模型：

```
from vllm import LLM
model = LLM("./Meta-Llama-3-8B-Instruct-FP8-Dynamic")
model.generate("Hello my name is")
```

使用lm_eval评估准确性（例如在gsm8k的250个样本上）：

注意

量化模型可能对bos token的存在较为敏感。lm_eval默认不添加bos token，因此在运行评估时，请确保包含 `add_bos_token=True` 参数。

```
$ MODEL=$PWD/Meta-Llama-3-8B-Instruct-FP8-Dynamic
$ lm_eval \
  --model vllm \
  --model_args pretrained=$MODEL,add_bos_token=True \
  --tasks gsm8k --num_fewshot 5 --batch_size auto --limit 250
```

以下是生成的分数示例：

Tasks	Version	Filter	n-shot	Metric	Value	Stderr
gsm8k	3	flexible-extract	5	exact_match↑	0.768±	0.0268
		strict-match	5	exact_match↑	0.768±	0.0268

7.4. 故障排除与支持

如果遇到任何问题或有功能请求，请在vllm-project/llm-compressor的GitHub仓库中提交问题。

7.5. 已弃用的流程

注意

以下信息保留供参考和搜索使用。下面描述的量化方法已弃用，推荐使用上文提到的llmcompressor方法。

对于静态按张量离线量化到FP8，请安装AutoFP8库。

```
git clone https://github.com/neuralmagic/AutoFP8.git
pip install -e AutoFP8
```

这个包引入了AutoFP8ForCausalLM和BaseQuantizeConfig对象，用于管理模型压缩的方式。

7.6. 使用静态激活缩放因子的离线量化

您可以使用AutoFP8和校准数据，通过启用 `activation_scheme="static"` 参数，为权重和激活值生成按张量的静态缩放因子。


```

from datasets import load_dataset
from transformers import AutoTokenizer
from auto_fp8 import AutoFP8ForCausalLM, BaseQuantizeConfig

pretrained_model_dir = "meta-llama/Meta-Llama-3-8B-Instruct"
quantized_model_dir = "Meta-Llama-3-8B-Instruct-FP8"

tokenizer = AutoTokenizer.from_pretrained(pretrained_model_dir, use_fast=True)
tokenizer.pad_token = tokenizer.eos_token

# Load and tokenize 512 dataset samples for calibration of activation scales
ds = load_dataset("mgoin/ultrachat_2k", split="train_sft").select(range(512))
examples = [tokenizer.apply_chat_template(batch["messages"], tokenize=False) for batch in
             ds]
examples = tokenizer(examples, padding=True, truncation=True, return_tensors="pt").to("cuda")

# Define quantization config with static activation scales
quantize_config = BaseQuantizeConfig(quant_method="fp8", activation_scheme="static")

# Load the model, quantize, and save checkpoint
model = AutoFP8ForCausalLM.from_pretrained(pretrained_model_dir, quantize_config)
model.quantize(examples)
model.save_quantized(quantized_model_dir)

```

您的模型检查点，包含量化后的权重和激活值，应位于Meta-Llama-3-8B-Instruct-FP8/。最后，您可以直接在vLLM中加载该量化模型检查点。

```

from vllm import LLM
model = LLM(model="Meta-Llama-3-8B-Instruct-FP8/")
# INFO 06-10 21:15:41 model_runner.py:159] Loading model weights took 8.4596 GB
result = model.generate("Hello, my name is")

```

8. 量化KV缓存

8.1. FP8 KV缓存

将KV缓存量化为FP8可以减少其内存占用。这会增加缓存中可以存储的令牌数量，从而提高吞吐量。

FP8格式

OCP（开放计算项目）指定了两种常见的8位浮点数据格式：

- E5M2（5个指数位和2个尾数位）
- E4M3FN（4个指数位和3个尾数位，通常简称为E4M3）

E4M3格式相比于E5M2提供了更高的精度。然而，由于其较小的动态范围（ ± 240.0 ），E4M3通常需
要与每个量化张量一起使用更高精度（FP32）的缩放因子。

8.1.1 FP8格式

OCP（开放计算项目）指定了两种常见的8位浮动点数据格式：

- E5M2（5个指数位和2个尾数位）
- E4M3FN（4个指数位和3个尾数位，通常简称为E4M3）

E4M3格式相比于E5M2提供了更高的精度。然而，由于其较小的动态范围（ ± 240.0 ），E4M3通常需
要与每个量化张量一起使用更高精度（FP32）的缩放因子。

8.1.2 当前限制

目前，仅支持按张量（标量）缩放因子。开发工作正在进行，以支持更细粒度的缩放因子（例如按通
道缩放）。

8.1.3 性能影响

当前的FP8 KV缓存实现主要通过允许大约双倍的KV缓存分配空间来提高吞吐量。这使得以下两种情
况成为可能：

- 处理更长的上下文长度以应对单个请求，或
- 处理更多的并发请求批次

然而，目前并没有提高延迟，因为该实现尚未包含融合的去量化和注意力操作。未来的版本将支持量
化的注意力操作，并结合硬件加速，这应当能提供额外的性能提升。虽然最新的硬件（例如AMD
MI300、NVIDIA Hopper及以后的型号）支持FP8与其他格式（如fp32、fp16、bf16）之间的原生硬件
转换，但这一优势尚未完全实现。

研究表明，FP8 E4M3量化通常只会对推理准确性产生最小的影响，因此它是吞吐量优化的一个实际
选择。

8.2. 使用示例

以下是如何启用FP8量化的示例：

```
#To calculate kv cache scales on the fly enable the calculate_kv_scales
# parameter

from vllm import LLM, SamplingParams

sampling_params = SamplingParams(temperature=0.7, top_p=0.8)
llm = LLM(model="meta-llama/Llama-2-7b-chat-hf",
          kv_cache_dtype="fp8",
          calculate_kv_scales=True)
prompt = "London is the capital of"
out = llm.generate(prompt, sampling_params)[0].outputs[0].text
print(out)
```

kv_cache_dtype 参数指定了KV缓存存储的数据类型：

- “auto”：使用模型的默认“未量化”数据类型
- “fp8” 或 “fp8_e4m3”：支持CUDA 11.8+和ROCm（AMD GPU）
- “fp8_e5m2”：支持CUDA 11.8+

8.3. 为了更好的准确性，使用校准的缩放因子

为了在使用FP8 KV缓存时获得最佳的模型质量，我们建议使用针对代表性推理数据调整的校准缩放因子。LLM Compressor是推荐的工具来完成这一过程。

8.3.1 安装

首先，安装所需的依赖项：

```
pip install llmcompressor
```

8.3.2 示例用法

这是一个使用 meta-llama/Llama-3.1-8B-Instruct 的完整示例（大多数模型可以使用相同的模式）：

```
from datasets import load_dataset
from transformers import AutoModelForCausalLM, AutoTokenizer
from llmcompressor.transformers import oneshot

# Select model and load it
MODEL_ID = "meta-llama/Llama-3.1-8B-Instruct"
model = AutoModelForCausalLM.from_pretrained(MODEL_ID, device_map="auto", torch_dtype="auto")
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)

# Select calibration dataset
DATASET_ID = "HuggingFaceH4/ultrachat_200k"
DATASET_SPLIT = "train_sft"
```

```

# Configure calibration parameters
NUM_CALIBRATION_SAMPLES = 512 # 512 samples is a good starting point
MAX_SEQUENCE_LENGTH = 2048

# Load and preprocess dataset
ds = load_dataset(DATASET_ID, split=DATASET_SPLIT)
ds = ds.shuffle(seed=42).select(range(NUM_CALIBRATION_SAMPLES))

def process_and_tokenize(example):
    text = tokenizer.apply_chat_template(example["messages"], tokenize=False)
    return tokenizer(
        text,
        padding=False,
        max_length=MAX_SEQUENCE_LENGTH,
        truncation=True,
        add_special_tokens=False,
    )

ds = ds.map(process_and_tokenize, remove_columns=ds.column_names)

# Configure quantization settings
recipe = """
quant_stage:
  quant_modifiers:
    QuantizationModifier:
      kv_cache_scheme:
        num_bits: 8
        type: float
        strategy: tensor
        dynamic: false
        symmetric: true
"""

# Apply quantization
oneshot(
    model=model,
    dataset=ds,
    recipe=recipe,
    max_seq_length=MAX_SEQUENCE_LENGTH,
    num_calibration_samples=NUM_CALIBRATION_SAMPLES,
)

# Save quantized model
SAVE_DIR = MODEL_ID.split("/")[1] + "-FP8-KV"
model.save_pretrained(SAVE_DIR, save_compressed=True)
tokenizer.save_pretrained(SAVE_DIR)

```

上述脚本将在当前目录中创建一个包含量化模型（例如，Llama-3.1-8B-Instruct-FP8-KV）和校准缩放因子的文件夹。

运行模型时，必须指定 `kv_cache_dtype="fp8"` 以启用KV缓存量化并使用这些缩放因子。

```
from vllm import LLM, SamplingParams

sampling_params = SamplingParams(temperature=0.7, top_p=0.8)
llm = LLM(model="Llama-3.1-8B-Instruct-FP8-KV", kv_cache_dtype="fp8")
prompt = "London is the capital of"
out = llm.generate(prompt, sampling_params)[0].outputs[0].text
print(out)
```